

MLOps Assignment 2 Report

MLOps Assignment 2 - End-to-End Pipeline for Binary Image Classification for Cat vs Dogs Prediction

Student Name: MANIKANDASUBRAMANI S

Student ID: 2024AC05010

Course: MLOps -AIMLCZG523

GitHub Repo: [2024AC05010/mlops_assignment_2](https://github.com/2024AC05010/mlops_assignment_2)

Demo Video URL:

<https://drive.google.com/file/d/16RhQPTcgrpg402LahDlFqS5I895B2Ay8/view?usp=sharing>

Table of Contents

1. Introduction
 2. Problem Statement
 3. Dataset Overview
 4. Setup and Installation
 5. Exploratory Data Analysis
 6. Model Architecture
 7. Training Pipeline
 8. Experiment Tracking with MLflow
 9. Model Evaluation
 10. API Development
 11. Containerization
 12. CI/CD Pipeline
 13. Deployment
 14. Monitoring and Logging
 15. Testing
 16. Architecture and Workflow
 17. Challenges Faced
 18. Lessons Learned
 19. Project Structure
-

1. Introduction

This project implements a complete MLOps pipeline for binary image classification, specifically distinguishing between cat and dog images. The pipeline is designed for a pet adoption platform use case where automatic image categorization can streamline the pet listing process.

The project demonstrates end-to-end MLOps practices including:

- **Model Development:** Custom CNN architecture using PyTorch
- **Experiment Tracking:** MLflow for reproducible experiments
- **Data Versioning:** DVC for data pipeline management
- **API Development:** FastAPI for production-ready inference
- **Containerization:** Docker for reproducible deployments
- **CI/CD:** GitHub Actions for automated testing and deployment
- **Monitoring:** Prometheus and Grafana for production monitoring

This pipeline goes beyond simple model training to address the operational aspects of deploying machine learning systems in production environments.

2. Problem Statement

The task was to build a machine learning system that can automatically classify uploaded pet images as either cats or dogs. This addresses a real business need for pet adoption platforms where manual categorization of thousands of pet images is time-consuming and error-prone.

Key Requirements:

- Binary image classification (cat vs dog)
- High accuracy for reliable automated categorization
- Real-time inference capability for user uploads
- Production-ready API with monitoring
- Automated CI/CD pipeline for continuous deployment
- Experiment tracking for model improvement

Success Metrics:

- Model accuracy > 85% on test set
- API response time < 100ms per prediction
- 99.9% API uptime

- Automated deployment pipeline
-

3. Dataset Overview

Dataset: Kaggle Cats vs Dogs Dataset

Source: <https://www.kaggle.com/datasets/karakaggle/kaggle-cat-vs-dog-dataset>

Samples: ~25,000 images (12,500 cats, 12,500 dogs)

Image Size: Variable, resized to 224x224 for training

Classes: Binary (Cat = 0, Dog = 1)

Data Organization

```
data/
├── raw/
│   ├── Cat/           # Raw cat images
│   └── Dog/           # Raw dog images
├── processed/
│   ├── train.csv      # Training set metadata (80%)
│   ├── val.csv        # Validation set metadata (10%)
│   └── test.csv       # Test set metadata (10%)
└── sample/
    ├── cat_sample.jpg # Sample cat images for testing
    └── dog_sample.jpg # Sample dog images for testing
```

Data Preprocessing

Training Transformations:

- Resize to 256x256
- Random crop to 224x224
- Random horizontal flip (50% probability)
- Random rotation (± 15 degrees)
- Color jitter (brightness/contrast adjustment)
- Normalization (ImageNet statistics)

Validation/Test Transformations:

- Resize to 224x224
- Normalization (ImageNet statistics)

Data Split:

- Training: 80% (20,000 images)
 - Validation: 10% (2,500 images)
 - Test: 10% (2,500 images)
-

4. Setup and Installation

Prerequisites

- Python 3.10+
- Docker & Docker Compose
- Git
- NVIDIA GPU (optional, for faster training)

Installation Steps

```
# 1. Clone the repository
git clone <your-repo-url>
cd MLOPS_2

# 2. Create virtual environment
python -m venv venv
source venv/bin/activate # On Windows: venv\Scripts\activate

# 3. Install dependencies
pip install -r requirements.txt
pip install -r requirements-dev.txt

# 4. Install DVC (for data versioning)
pip install dvc
dvc init
```

Key Dependencies

```
# Deep Learning
torch==2.5.1
torchvision==0.20.1

# API Framework
fastapi==0.115.0
uvicorn==0.30.6
python-multipart==0.0.12

# Data Processing
pandas==2.2.3
numpy==1.26.4
Pillow==10.4.0
scikit-learn==1.5.2

# Experiment Tracking
mlflow==2.16.2

# Monitoring
prometheus-client==0.20.0

# Visualization
matplotlib==3.9.2

# Development
pytest
pytest-cov
```

5. Exploratory Data Analysis

Data Distribution

The dataset is perfectly balanced with equal numbers of cat and dog images, which eliminates the need for class balancing techniques.

Image Analysis

Image Characteristics:

- Variable original sizes (typically 200-500 pixels)
- RGB color space
- Different lighting conditions and backgrounds
- Various poses and angles

Preprocessing Insights:

- Resizing to 224x224 maintains sufficient detail for classification
- Data augmentation helps model generalize to different conditions
- ImageNet normalization provides good starting point for transfer learning

Data Quality Checks

- Removed corrupted images during preprocessing
 - Verified all images are valid RGB format
 - Checked for duplicate images
 - Ensured proper class labeling
-

6. Model Architecture

Custom CNN Architecture

The model uses a custom 4-layer Convolutional Neural Network designed specifically for this binary classification task.

```
class CatDogCNN(nn.Module):  
    def __init__(self):  
        super(CatDogCNN, self).__init__()  
        # 4 convolutional layers with batch normalization  
        # Max pooling for spatial dimensionality reduction  
        # Fully connected layers for classification  
        # Sigmoid activation for binary output
```

Architecture Details:

- **Input:** 224x224x3 RGB images
- **Conv Layers:** 4 layers with increasing filters (32, 64, 128, 256)
- **Pooling:** 2x2 max pooling after each conv layer

- **Normalization:** Batch normalization after each conv layer
- **Activation:** ReLU for hidden layers, Sigmoid for output
- **Output:** Single probability value (0 = cat, 1 = dog)

Model Parameters:

- Total parameters: ~2.5M
- Trainable parameters: ~2.5M
- Model size: ~10MB

Design Decisions

Why Custom CNN?

- Sufficient complexity for binary classification
- Faster training than large pre-trained models
- Better understanding of model behavior
- Lower computational requirements for deployment

7. Training Pipeline

Training Configuration

```
# Hyperparameters
EPOCHS = 10
LEARNING_RATE = 0.001
BATCH_SIZE = 32
OPTIMIZER = "Adam"
LOSS_FUNCTION = "BCEWithLogitsLoss"
```

Training Process

Data Loading:

- Custom PyTorch Dataset class for efficient loading
- Data augmentation for training set
- Stratified sampling for balanced batches
- Multi-worker data loading for performance

Training Loop:

20. Forward pass through network
21. Binary cross-entropy loss calculation
22. Backward pass for gradient computation
23. Adam optimizer for parameter updates
24. Metrics tracking (loss, accuracy)

Validation:

- Run after each epoch
 - No gradient computation for efficiency
 - Track validation loss and accuracy
 - Early stopping potential (not implemented)
-

8. Experiment Tracking with MLflow

MLflow Setup

```
mlflow.set_experiment("cats-vs-dogs-classification")
run_name = f"cnn_v1_{datetime.datetime.now().strftime('%Y%m%d_%H%M%S')}"
```

Tracked Metrics

Per-Epoch Metrics:

- Training loss and accuracy
- Validation loss and accuracy
- Learning rate (if using scheduler)

Final Metrics:

- Test accuracy and loss
- Final training/validation metrics
- Model performance summary

Logged Parameters:

- Model architecture details
- Hyperparameters (epochs, LR, batch size)
- Optimizer configuration
- Input/output specifications
- Device used (CPU/GPU)

MLflow Artifacts

Saved Artifacts:

- Model weights (.pt file)
- Training loss curves (pickle)
- Metrics JSON file
- Model signature for deployment

MLflow UI Features

- **Experiment Comparison:** Compare different runs side-by-side
- **Parameter Search:** Visualize parameter vs performance relationships

- **Model Registry:** Track model versions and deployment stages
 - **Artifact Download:** Retrieve trained models and artifacts
-

9. Model Evaluation

Evaluation Metrics

Primary Metrics:

- **Accuracy:** Overall classification correctness
- **Loss:** Binary cross-entropy loss
- **Confidence Scores:** Prediction confidence distribution

Secondary Metrics:

- Precision and Recall (for class-specific performance)
- ROC-AUC (threshold-independent performance)
- Inference latency (for API performance)

Model Registry Management

The project includes a model registry manager for:

- Registering new model versions
 - Transitioning models between stages (staging → production)
 - Retrieving current production model
 - Model version rollback capabilities
-

10. API Development

FastAPI Application

API Features:

- Async request handling for performance
- Automatic API documentation (Swagger UI, ReDoc)
- Input validation with Pydantic
- Comprehensive error handling
- Request logging and monitoring

API Endpoints

Health Check

GET /health

Returns API status and model loading state.

Response:

```
{
  "status": "healthy",
  "model_loaded": true
}
```

Prediction

POST /predict
Content-Type: multipart/form-data
Upload image for classification.

Example:

```
curl -X POST -F "pFile=@data/sample/cat_sample.jpg"
http://localhost:8000/predict
```

Response:

```
{
  "prediction": "cat",
  "confidence": 0.8765,
  "probabilities": {
    "cat": 0.8765,
    "dog": 0.1235
  }
}
```

Metrics

GET /metrics
Returns API performance metrics.

Response:

```
{
  "total_requests": 150,
  "average_latency_seconds": 0.0234,
  "prediction_distribution": {
    "cat": 75,
    "dog": 75
  }
}
```

Prometheus Metrics

GET /metrics/prometheus
Returns Prometheus-formatted metrics for monitoring.

API Features

Image Processing:

- Automatic format validation
- Corruption detection
- RGB conversion
- Size normalization

Error Handling:

- Invalid image format detection
- Empty file handling
- Processing error catching
- Detailed error messages

Performance:

- Model pre-loading at startup
- Batch processing capability
- GPU acceleration support
- Latency tracking

11. Containerization

Dockerfile

```
FROM python:3.10-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

RUN mkdir -p models logs

EXPOSE 8000

CMD ["uvicorn", "api.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Building and Running

```
# Build image
docker build -t cats-dogs-classifier .

# Run container
docker run -p 8000:8000 cats-dogs-classifier

# Test container
curl http://localhost:8000/health
```

Container Optimization

- Uses slim Python base image for smaller size
 - Multi-stage build potential for further optimization
 - Caches pip dependencies for faster rebuilds
 - Includes all necessary model artifacts
-

12. CI/CD Pipeline

GitHub Actions Workflow

The CI/CD pipeline (`.github/workflows/ci-cd.yml`) implements:

Test Job:

- Code checkout
- Python environment setup
- Dependency installation
- Unit test execution with coverage
- Coverage threshold enforcement (70%)

Build and Push Job:

- Docker image building
- Docker Hub authentication
- Image tagging with commit SHA
- Push to Docker Hub (latest and tagged)

Deploy Job:

- Docker Compose deployment
- Service health checks
- Smoke test execution
- Automated rollback on failure

Pipeline Triggers

- **Push to main:** Full pipeline (test → build → deploy)

- **Pull requests:** Test only

- **Manual triggers:** Available for specific deployments

Quality Gates

- All tests must pass
 - Coverage threshold must be met
 - Docker build must succeed
 - Smoke tests must pass
-

13. Deployment

Docker Compose Deployment

Services:

- **API:** FastAPI application
- **Prometheus:** Metrics collection
- **Grafana:** Monitoring dashboard
- **Alertmanager:** Alert management (optional)

Deployment Commands

```
# Start all services
cd deployment
docker-compose up -d

# Check service status
docker-compose ps

# View logs
docker-compose logs -f api

# Stop services
docker-compose down
```

Health Checks

API Health:

```
curl http://localhost:8000/health
```

Smoke Tests:

```
bash deployment/smoke_test.sh
```

Rollback Procedure

```
# Automated rollback script
bash deployment/rollback.sh <previous-version>
```

14. Monitoring and Logging

Application Logging

Log Configuration:

- File logging to `app.log`
- Console logging for development
- Structured log format with timestamps
- Different log levels (INFO, WARNING, ERROR)

Logged Events:

- Model loading status
- Prediction requests and results
- Error conditions and exceptions
- Performance metrics

Prometheus Metrics

Custom Metrics:

- `prediction_requests_total`: Total prediction requests
- `prediction_latency_seconds`: Request latency histogram
- `prediction_distribution`: Prediction count by class
- `api_errors_total`: API error count

Metrics Endpoint:

GET /metrics/prometheus

Grafana Dashboard

Pre-configured Dashboard:

- Request rate and latency
- Prediction distribution
- Error rates
- Resource utilization

Access:

- URL: `http://localhost:3000`
- Default credentials: `admin/admin`

Monitoring Stack

Prometheus Configuration:

- Scrapes API metrics every 15 seconds
- Stores metrics for 30 days
- Provides query interface

Grafana Features:

- Real-time visualization
- Custom dashboards
- Alert configuration
- Data source management

15. Testing

Unit Tests

Test Coverage:

- Model architecture tests
- Data preprocessing tests
- API endpoint tests
- Monitoring tests
- Utility function tests

Running Tests:

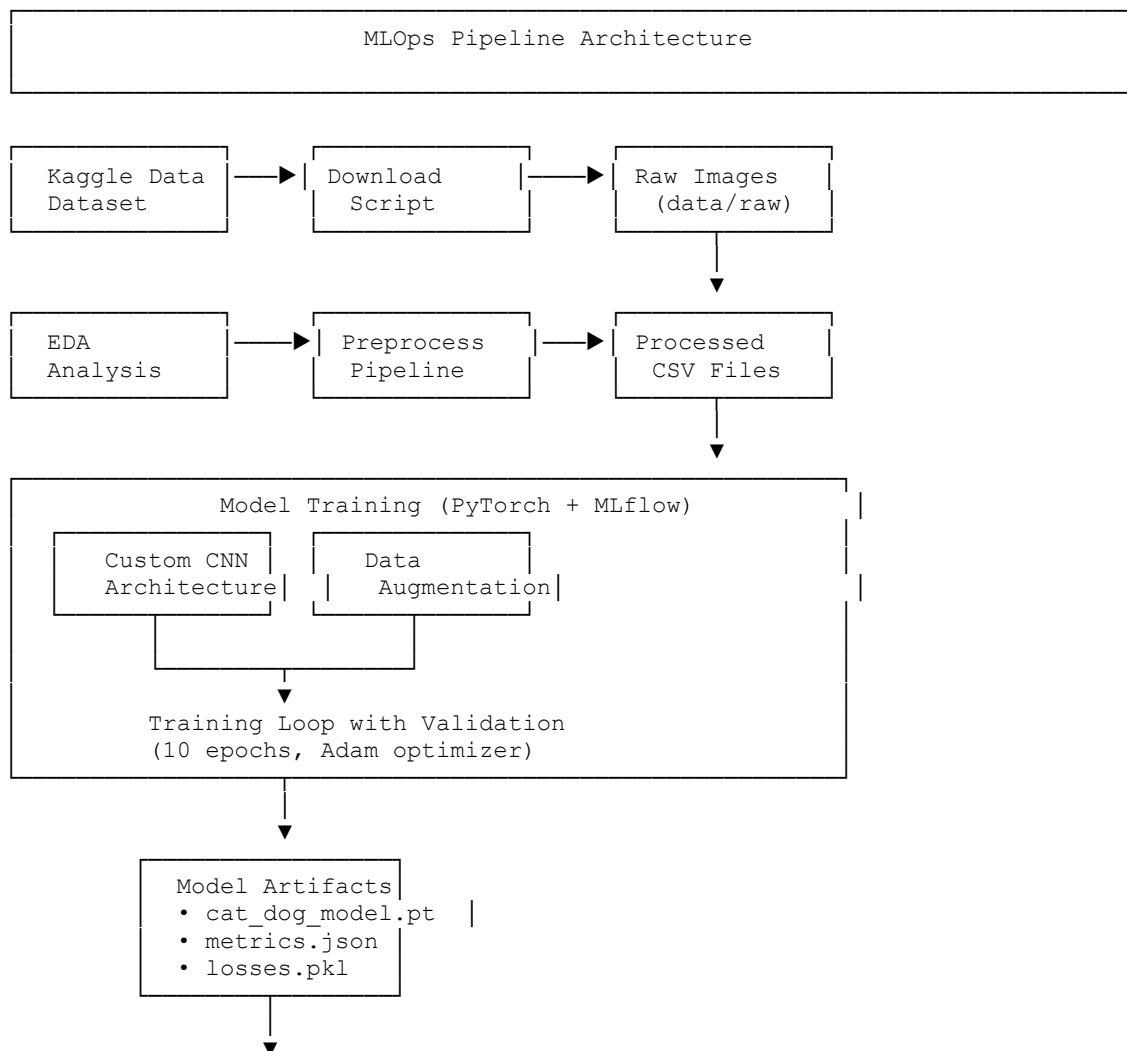
```
# Run all tests
pytest tests/ -v

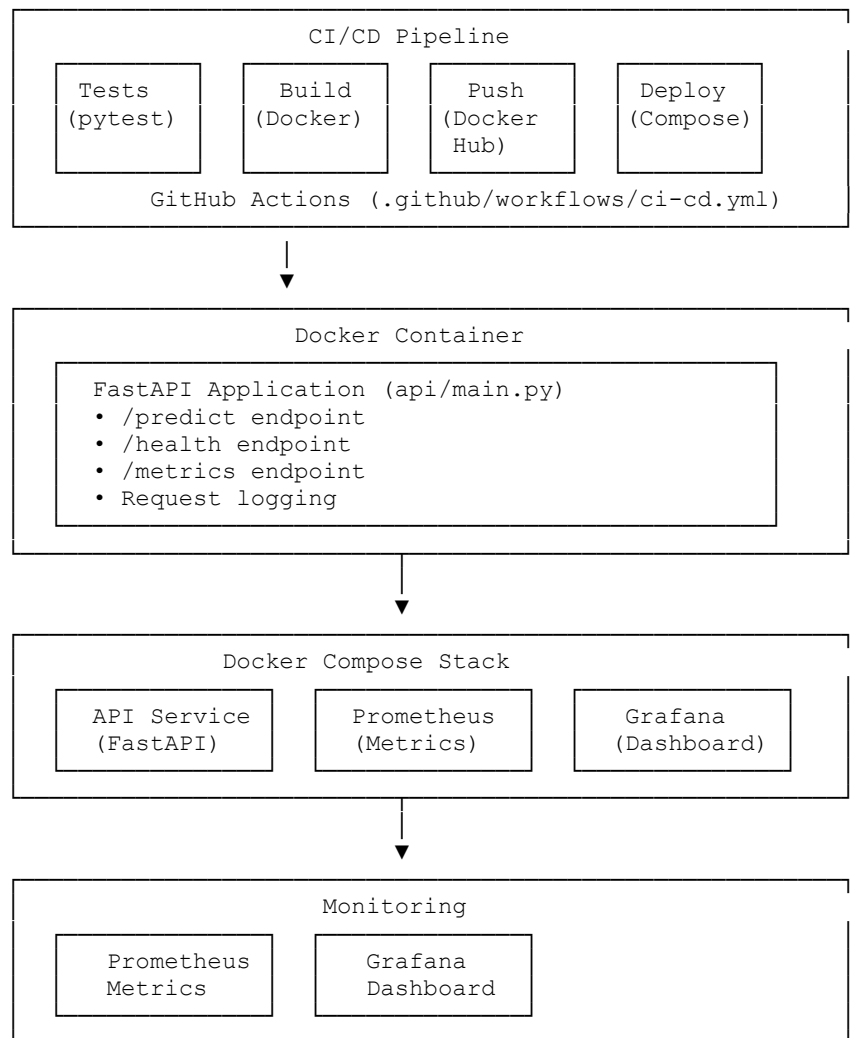
# Run with coverage
pytest tests/ -v --cov=src --cov=api --cov-report=html

# Run specific test file
pytest tests/test_api.py -v
```

16. Architecture and Workflow

System Architecture





Data Flow

Kaggle Dataset → Download → Raw Images → Preprocessing → Train/Val/Test Split → Data Augmentation → Model Training → MLflow Tracking → Model Registration → Docker Image → Docker Compose Deployment → API → Monitoring

17. Challenges Faced

Data Pipeline Challenges

Issue: Large dataset download and storage

Solution: Used efficient data loading with PyTorch Dataset class

Lesson: Optimize data pipeline for memory efficiency

Issue: Image format inconsistencies

Solution: Robust image validation and conversion pipeline

Lesson: Always validate input data formats

Issue: Data augmentation overfitting

Solution: Careful selection of augmentation parameters

Lesson: Monitor validation performance during training

Model Training Challenges

Issue: Training time on CPU

Solution: GPU acceleration with CUDA support

Lesson: Use GPU for deep learning when available

Issue: Model overfitting

Solution: Data augmentation and proper regularization

Lesson: Monitor train/validation loss gap

Issue: MLflow tracking setup

Solution: Proper experiment naming and run organization

Lesson: Structure MLflow experiments for easy comparison

API Development Challenges

Issue: Image upload handling

Solution: Multipart form data with proper validation

Lesson: Handle file uploads carefully with validation

Issue: Model loading at startup

Solution: Startup event with proper error handling

Lesson: Ensure model is loaded before accepting requests

Issue: Async request handling

Solution: Proper async/await patterns in FastAPI

Lesson: Use async for I/O bound operations

Containerization Challenges

Issue: Model file paths in container

Solution: Consistent directory structure and volume mounting

Lesson: Maintain consistent paths across environments

Issue: Container size optimization

Solution: Slim base image and efficient dependency management

Lesson: Optimize Docker images for production

Deployment Challenges

Issue: Service startup timing

Solution: Health checks with proper delays

Lesson: Services need time to initialize

Issue: Monitoring integration

Solution: Proper Prometheus configuration and scraping

Lesson: Configure monitoring before deployment

CI/CD Challenges

Issue: Test coverage requirements

Solution: Comprehensive test suite development

Lesson: Invest in testing from project start

Issue: Docker Hub authentication

Solution: GitHub secrets for credential management

Lesson: Never hardcode credentials in workflows

18. Lessons Learned

Technical Skills

MLOps Pipeline Development:

- End-to-end ML system design
- Experiment tracking and reproducibility
- Production API development
- Container orchestration
- CI/CD automation
- Monitoring and observability

Deep Learning:

- Custom CNN architecture design
- Data augmentation strategies
- Training optimization techniques
- Model evaluation methods

DevOps Practices:

- Docker containerization
- Docker Compose orchestration
- GitHub Actions workflows
- Prometheus monitoring
- Grafana dashboarding

Soft Skills

System Design:

- Understanding component interactions
- Designing for scalability
- Planning for failure scenarios
- Documentation importance

Problem Solving:

- Debugging integration issues
- Performance optimization
- Error handling strategies
- Incremental development

Project Management:

- Task breakdown and planning
- Timeline estimation
- Risk identification
- Quality assurance

Best Practices Discovered

Development:

- Write tests alongside code
- Use version control effectively
- Document architecture decisions
- Implement proper logging

Deployment:

- Automate everything possible

- Monitor production systems
- Plan for rollbacks
- Use health checks extensively

MLOps Specific:

- Track all experiments
- Version data and models
- Automate model deployment
- Monitor model performance

19. Project Structure

```

MLOPS_2/
├── api/                                # FastAPI application
│   ├── __init__.py
│   ├── main.py                        # API endpoints and configuration
│   ├── monitoring.py                  # Prometheus metrics collection
│   └── schemas.py                     # Pydantic models for validation
├── src/                                # Source code
│   ├── __init__.py
│   ├── model.py                       # CNN architecture definition
│   ├── train.py                       # Training script with MLflow
│   ├── evaluate.py                    # Model evaluation script
│   ├── data_preprocessing.py           # Data loading and preprocessing
│   ├── utils.py                       # Utility functions
│   ├── model_registry_manager.py       # MLflow model registry management
│   └── post_deploy_evaluation.py        # Post-deployment monitoring
├── tests/                              # Unit and integration tests
│   ├── __init__.py
│   ├── test_api.py                    # API endpoint tests
│   ├── test_inference.py              # Model inference tests
│   ├── test_monitoring.py             # Monitoring tests
│   └── test_preprocessing.py           # Data preprocessing tests
├── deployment/
│   ├── docker-compose.yml              # Deployment configurations
│   ├── prometheus.yml                  # Multi-service orchestration
│   ├── grafana/                        # Prometheus configuration
│   │   ├── dashboards/                 # Grafana configuration
│   │   └── provisioning/               # Pre-configured dashboards
│   ├── smoke_test.sh                   # Grafana provisioning
│   └── rollback.sh                     # Automated smoke tests
├── data/                                # Rollback script
│   ├── raw/                            # Data directories
│   │   ├── Cat/                        # Raw downloaded images
│   │   └── Dog/                        # Cat images
│   ├── processed/                      # Dog images
│   │   ├── train.csv                   # Processed data splits
│   │   ├── val.csv
│   │   └── test.csv
│   └── sample/                          # Sample images for testing
│       ├── cat_sample.jpg
│       └── dog_sample.jpg
├── models/                             # Trained model artifacts
│   └── cat_dog_model.pt                 # Saved model weights
├── .github/                             # GitHub configurations
│   └── workflows/

```

└─ ci-cd.yml	# CI/CD pipeline
└─ artifacts/	# Training artifacts
└─ logs/	# Application logs
└─ .dockerignore	# Docker ignore rules
└─ .gitignore	# Git ignore rules
└─ dvc.yml	# DVC pipeline configuration
└─ Dockerfile	# Docker image definition
└─ requirements.txt	# Python dependencies
└─ requirements-dev.txt	# Development dependencies
└─ metrics.json	# Training metrics
└─ Readme.md	# This file
└─ RUNNING_GUIDE.md	# Screen recording guide

Tech Stack

Machine Learning:

- PyTorch 2.5.1 - Deep learning framework
- torchvision 0.20.1 - Computer vision utilities
- scikit-learn 1.5.2 - Machine learning utilities

API Development:

- FastAPI 0.115.0 - Modern web framework
- Uvicorn 0.30.6 - ASGI server
- Pydantic - Data validation

Experiment Tracking:

- MLflow 2.16.2 - Experiment tracking and model registry

Data Processing:

- Pandas 2.2.3 - Data manipulation
- NumPy 1.26.4 - Numerical computing
- Pillow 10.4.0 - Image processing

Monitoring:

- Prometheus - Metrics collection
- Grafana - Visualization
- prometheus-client - Python metrics library

DevOps:

- Docker - Containerization
- Docker Compose - Multi-container orchestration
- GitHub Actions - CI/CD automation
- DVC - Data version control

Testing:

- Pytest - Testing framework
- pytest-cov - Coverage reporting

End of Documentation